

Automated Code-Guided Chart Synthesis and Visual Degradation: A Unified Framework for Multimodal Document Parsing and Visual Reasoning Benchmarks

Abstract

Modern multimodal large language models (MLLMs) and visual document parsing networks require massive, structurally diverse datasets to decode the complex relationships between text, categorical structures, and graphical primitives. Crowdsourced human annotation pipelines are non-scalable, labor-intensive, and prone to low-level spatial coordinate errors. This paper introduces an automated, code-guided chart synthesis framework that connects explicit mathematical data-generation engines directly to high-level graphical layout compilers. Operating across specialized scientific and business domains, our system generates highly realistic bar, line, box plot, heatmap, pie, histogram, scatter, and area charts while extracting pixel-perfect ground-truth coordinates, visual scene graphs, and adaptive multi-keypoint pose schemas. To minimize the visual domain gap between idealized digital graphics and real-world scanned documents, vector outputs are passed through a comprehensive post-rendering physical degradation pipeline. This pipeline executes perspective homography transforms, multi-channel chromatic aberrations, spatial blur kernels, and non-uniform lighting fields, while drawing on Tufte's (2001) design philosophy of graphical integrity by minimizing non-data ink and analyzing coordinate-space prerequisites for non-linear scale verification.

Keywords: Code-Guided Synthesis, Multimodal Large Language Models, Visual Encodings, Computational Visualization, Image Degradation Frameworks, Spatial Ground Truth.

1. Introduction

The interpretation of data visualizations stands as a critical benchmark for evaluating visual reasoning capabilities in machine learning models, requiring networks to combine textual optical character recognition (OCR), structural orientation tracking, coordinate scale matching, and multi-step quantitative reasoning. Traditional computer vision datasets rely extensively on crowdsourced manual annotations. This paradigm is inherently non-scalable and prone to bounding box inaccuracies, which can severely disrupt the training of networks designed to extract precise numerical values from high-density charts.

To circumvent the limits of human labeling, modern chart visual question answering (VQA) and multimodal benchmarks utilize code-guided visualization synthesis pipelines. By programmatically linking data frames to visual styles, these frameworks establish a direct, automated channel between data tables and graphic components. This approach forms the foundation of modern vision-language benchmarks like **ChartNet** (Kondic et al., 2026), **ChartMaster** (Liu et al., 2026), and **ChartGemma** (Masry et al., 2024), which convert raw databases into executable rendering configurations to build visual reasoning models.

However, training document understanding models purely on clean digital vectors introduces a significant domain gap. Real-world documents encounter physical print-and-scan alterations, hand-held camera perspective warping, lens defocusing, and sensor noise. To address this discrepancy, synthetic visualization pipelines must integrate post-rendering degradation operations that simulate physical paper defects while dynamically keeping all internal spatial coordinate logs updated.

This article details a comprehensive, code-guided visualization synthesis and image degradation architecture drawing on Tufte's (2001) design philosophy of graphical integrity. Specifically, our minimal

and prism themes reduce non-data ink, while we analyze the coordinate-space prerequisites for accurate Lie Factor implementation on non-linear axes. We examine how these design constraints execute across high-level statistical plotting architectures like Matplotlib (Hunter, 2007) and Seaborn (Waskom, 2021).

Finally, we present a technical evaluation of our computational pipeline. We establish a formal mathematical record of the system's design parameters, analyze semantic representations in multi-dimensional serialization channels, and document the coordinate-space prerequisite transformations required to implement non-linear scales. By providing explicit mathematical models, structural taxonomy pools, and geometric warp profiles, this work provides a rigorous foundation for building robust, submission-ready visual document reasoning benchmarks.

2. Pipeline Architecture & Domain Taxonomy

The generation framework configures the chart rendering workspace through a unified configuration hierarchy, splitting variables across two structural application profiles. This guarantees that categorical taxonomies, noise boundaries, and presentation themes mirror the distinct publishing rules of technical journals and corporate dashboards.

2.1 The Scientific Profile

The scientific profile emulates experimental measurements, biological assays, and clinical outcomes. The text engine selects string parameters from an index of over 100 professional measurement combinations (`SCIENTIFIC_DOMAIN_DICT`), dynamically injecting axis labels such as *Reads per million (RPM)*, *Binding Affinity (K_d , nM)*, *Optical Density (OD_{600})*, *Molarity (mM)*, *Fold Change*, and *NMR Chemical Shift (ppm)*.

Categorical markers pull from authentic biological sequences, using common oncogenes (`TP53` , `BRCA1` , `BRCA2` , `EGFR` , `KRAS`) and cellular pathways (`PI3K-AKT` , `MAPK` , `WNT` , `Apoptosis`). Visually, the scientific profile enforces high-precision error bars, explicit statistical significance brackets, and highly skewed or zero-inflated continuous density structures to model realistic physical instrumentation baselines.

2.2 The Business Profile

The business profile emulates corporate analytics, performance indicators, and economic charts. Text strings map enterprise metric taxonomies (`BUSINESS_DOMAIN_DICT`), utilizing combinations like *Monthly Active Users (MAU)*, *Customer Lifetime Value (LTV \$)*, *Cost per Acquisition (CAC \$)*, *Net Promoter Score (NPS)*, and *EBITDA (\$)*.

Categorical labels draw from organizational records, mapping specific product SKUs, subscription billing cycles, fulfillment timelines, and localized geographic sales regions. Structurally, the business profile focuses on multi-component seasonal trends, exponential adoption constraints, market ceilings, and clean monochromatic or corporate-accented color layouts.

3. Mathematical Data Generation Engines

Rather than utilizing flat uniform random variables, our framework draws continuous tracks from explicit mathematical equations parameterized to model realistic natural phenomena and business records.

3.1 Dose-Response Curves (The Hill Equation)

To model drug efficacy, chemical stimulus bounds, or cellular transitions across a range of log concentrations, the system samples a sigmoidal profile:

$$data = baseline + \frac{max_response - baseline}{1 + 10^{(ec50 - log_conc) \cdot hill_slope}}$$

Concentration bounds range realistically from pM to mM scales ($log_conc \in [-10, -3]$), while the inflection median scales as $ec50 \sim \mathcal{U}(-8.5, -4.5)$. The cooperativity coefficient $hill_slope$ varies dynamically based on physiological targets: $\mathcal{U}(0.7, 1.3)$ for standard single binding profiles, $\mathcal{U}(1.3, 2.0)$ for cooperative transitions, and $\mathcal{U}(2.0, 3.5)$ for high-cooperativity profiles.

Measurement noise is non-uniform, scaling dynamically with the local slope to incorporate heteroscedastic error modeling. The coefficient of variation (CV) peaks near the inflection thresholds of the sigmoidal response track:

$$cv = 0.05 + 0.10 \sqrt{response_fraction \cdot (1 - response_fraction)}$$

$$noise \sim \mathcal{N}(0, (data \cdot cv)^2)$$

3.2 Biological & Technical Replicates

To replicate variability across independent cell populations or testing cohorts, the pipeline avoids symmetric Gaussian noise, prioritizing an asymmetric log-normal distribution:

$$\sigma = \sqrt{\ln(1 + CV_{\text{target}}^2)}$$

$$\mu = \ln(mean_val) - 0.5\sigma^2$$

$$data \sim \exp(\mathcal{N}(\mu, \sigma^2))$$

The target parameter CV_{target} matches validated literature envelopes based on the selected testing modality (qPCR: 5–15%, Western Blot: 10–25%, dynamic cell microplates: 15–35%).

3.3 Exponential Decay and Pharmacokinetic Clearance

Chemical clearance, radioactive isotope degradation, or pharmacokinetic elimination parameters are simulated across a temporal vector (t) via an exponential decay model:

$$data = (initial_value - baseline) \cdot e^{-k \cdot t} + baseline$$

The clearance constant (k) scales from explicit target half-lives ($t_{1/2} \in [0.5, 72]$ hours):

$$k = \frac{\ln(2)}{half_life}$$

A proportional error envelope overlays the sequence to scale noise variance with current concentration levels:

$$noise \sim \mathcal{N}(0, (data \cdot 0.08 + 0.02 \cdot max_scale)^2)$$

3.4 Enzyme Kinetics (Michaelis-Menten Velocity Curve)

Initial reaction velocities of enzymatic processes under varying substrate concentrations are modeled via:

$$data = \frac{V_{\max} \cdot \text{substrate}}{K_m + \text{substrate}}$$

where $V_{\max}$ is the maximum reaction velocity and K_m is the Michaelis constant, representing the substrate concentration at half-maximum velocity. Measurement noise follows a normally distributed error proportional to stable assay variance with $CV \in [0.05, 0.15]$, consistent with validated analytical chemistry quality control bounds.

3.5 Spectroscopy & Chromatography Peaks (Gaussian Profile)

Sensor tracking streams from mass spectrometry or chromatography elution profiles are generated using a Gaussian peak model:

$$data = amplitude \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}} + baseline$$

To replicate authentic instrument physics, Poisson-like photon counting noise is injected as a function of the current signal magnitude above baseline:

$$noise \sim \mathcal{N}\left(0, \left(0.3\sqrt{|data - baseline|}\right)^2\right)$$

3.6 Multi-Component Seasonality & Trend

Macroeconomic timelines, consumer trends, and retail tracking lines are simulated across fiscal calendars using combined periodic functions:

$$data = base_level + (trend_slope \cdot x) + \sum_m [amp_m \cdot \cos(freq_m \cdot x + phase_m)]$$

The sub-component matrix m concurrently evaluates annual periods ($freq = \frac{2\pi}{12}$) and quarterly cycles ($freq = \frac{2\pi}{4}$), while noise variance expands proportionally during peak seasonal waves.

3.7 Saturated Exponential Growth

User adoption curves, SaaS subscriber acquisition tracks, and market entry sequences are bounded by an explicit logistic carrying capacity ceiling:

$$raw = initial_value \cdot e^{growth_rate \cdot t} \implies data = raw \cdot \left(\frac{1}{1 + \frac{raw}{carrying_capacity}} \right)$$

3.8 Pareto Distributions

Disparities across discrete operational brackets, regional performance metrics, or wealth concentrations are sampled via a power-law Pareto distribution:

$$data \sim \text{Pareto}(\alpha, x_m) \quad \text{where} \quad \alpha \in [1.05, 2.5]$$

3.9 Compositional Data Models (The Aitchison Simplex)

Proportional tracking charts represent data bounded on the Aitchison Simplex ($\mathcal{S}^D$), a restricted vector space where all elements must be strictly positive and sum exactly to 1:

$$\mathcal{S}^D = \left\{ \mathbf{x} = [x_1, x_2, \dots, x_D] \in \mathbb{R}^D \mid x_i > 0, \sum_{i=1}^D x_i = 1 \right\}$$

The algebraic operations used to generate and manipulate these compositions include:

- **Closure ($\mathcal{C}$):** Projects any raw positive vector into the simplex space by dividing each component by the cumulative sum:

$$\mathcal{C}(\mathbf{x}) = \left[\frac{x_1}{\sum_{k=1}^D x_k}, \frac{x_2}{\sum_{k=1}^D x_k}, \dots, \frac{x_D}{\sum_{k=1}^D x_k} \right]$$

- **Perturbation ($\oplus$):** Represents compositional addition on the simplex, performing component-wise multiplication followed by closure scaling:

$$\mathbf{x} \oplus \mathbf{y} = \mathcal{C}([x_1 y_1, x_2 y_2, \dots, x_D y_D])$$

- **Power Transform ($\odot$):** Multiplies a composition by a scalar, raising components to a power before projection:

$$\alpha \odot \mathbf{x} = \mathcal{C}([x_1^\alpha, x_2^\alpha, \dots, x_D^\alpha])$$

- **Aitchison Distance (d_A):** Computes the true log-ratio geometric distance between two compositions using the geometric mean $g(\mathbf{x}) = (\prod_{i=1}^D x_i)^{1/D}$:

$$d_A(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^D \left(\ln \frac{x_i}{g(\mathbf{x})} - \ln \frac{y_i}{g(\mathbf{y})} \right)^2}$$

Slices ($D \in [3, 12]$) are generated via several sampling configurations:

1. **Asymmetric Dirichlet Distribution:** $f(\mathbf{x}; \boldsymbol{\alpha}) = \frac{1}{B(\boldsymbol{\alpha})} \prod_{i=1}^D x_i^{\alpha_i - 1}$, modified by an asymmetry trend factor ($\text{trend} \in [1.0 - s, 1.0 + s]$, $s \in [0.0, 1.5]$).
2. **Stick-Breaking Process:** Generates sequential decay proportions using continuous draws from a Beta distribution to construct decreasing structural tail sizes:
 $V_k \sim \text{Beta}(1, \gamma)$, $\gamma \in [1.0, 8.0]$; $x_k = V_k \prod_{j=1}^{k-1} (1 - V_j)$.
3. **Kummer-Dirichlet-Gaussian-Asymmetric (KDGA) Model:** Integrates a Kummer-Dirichlet exponential tilting factor into Gamma random variables to form an explicit asymmetric exponential density layout:

$$f(\mathbf{x}) \propto \left(\prod_{i=1}^D x_i^{\alpha_i - 1} \right) \exp \left(-\lambda \sum_{i=1}^D x_i \right)$$

4. **Additive Logistic-Normal (ALN) Noise:** Perturbs vectors using the Additive Log-Ratio transform ($\text{alr}(\mathbf{x})$):

$$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I} \cdot \sigma^2) \quad \text{where} \quad \sigma \in [0.02, 0.08] \implies \mathbf{x}_{\text{noisy}} = \text{alr}^{-1}(\text{alr}(\mathbf{x}) + \epsilon)$$

When the number of slices fragments past a target threshold ($\text{max_slices} = 5$), low-weight slices are collapsed into a unified "Other" category governed by Shannon entropy tracking to limit detail loss:

$$H(\mathbf{x}) = - \sum_{i=1}^D x_i \log_2 x_i \implies \Delta H = H(\mathbf{x}_{\text{original}}) - H(\mathbf{x}_{\text{aggregated}})$$

If $\Delta H > \text{entropy_loss_tolerance}$ (0.5), the consolidation loops terminate to preserve fine-grained structural data.

3.10 Advanced Continuous and Zero-Inflated Profiles for Histograms

Histogram observation pools ($N \in [300, 1200]$) are sampled across continuous and zero-inflated frameworks to mimic instrumentation baselines:

- **Gaussian Mixture Models (GMM):** $f(x) = \sum_{k=1}^K w_k \cdot \frac{1}{\sigma_k \sqrt{2\pi}} \exp\left(-\frac{(x-\mu_k)^2}{2\sigma_k^2}\right)$, where components $K \in [2, 4]$ and weights $\mathbf{w}$ follow a uniform Dirichlet distribution.
- **Weibull Distribution:** $f(x) = \frac{c}{\lambda} \left(\frac{x}{\lambda}\right)^{c-1} \exp\left(-\left(\frac{x}{\lambda}\right)^c\right)$, with shape $c \in [0.8, 2.5]$.
- **Zero-Inflated Frameworks:** Introduces a structural zero probability factor ($\pi \in [0.2, 0.6]$) combined with a continuous or discrete peak:

$$\text{ZIG (Zero-Inflated Gaussian): } X_i = \begin{cases} 0 & \text{with probability } \pi \\ \mathcal{N}(\mu, \sigma^2) & \text{with probability } 1 - \pi \end{cases}$$

3.11 Scatter Trend and Clustered Models

Scatter tracks are mapped against target Coefficient of Determination (R^2) bounds to model specific linear and non-linear trends:

1. **Linear Core:** Fits a target vector $\mathbf{y}_{\text{perfect}} = m \cdot (\mathbf{x} - \bar{x}) + c$, then injects targeted Gaussian noise derived from the residual variance to satisfy the assigned R^2 window:

$$\sigma_{\text{noise}}^2 = \sigma_{\mathbf{y}_{\text{perfect}}}^2 \cdot \left(\frac{1 - R^2}{R^2}\right) \implies \mathbf{y}_i = \mathbf{y}_{\text{perfect},i} + \mathcal{N}(0, \sigma_{\text{noise}}^2)$$

2. **Nonlinear Tracks:** Maps quadratic ($\mathbf{y}_{\text{base}} = a\mathbf{x}^2 + b\mathbf{x} + c$), exponential ($\mathbf{y}_{\text{base}} = a \cdot \exp(b\mathbf{x}/\text{max_scale}) + c$), or logarithmic ($\mathbf{y}_{\text{base}} = a \cdot \ln(\mathbf{x} + 1) + c$) matrices with a proportional coefficient of variation noise layer ($\text{CV}_{\text{noise}} \in [0.08, 0.15]$).
3. **Bivariate Multicenter Profiles (Elliptical Mixtures):** Generates $K \in [2, 3, 4]$ independent data groups using multivariate normal distributions to form dense visual clouds:

$$\text{Cluster Size } N_k = \max(5, \lfloor N/K \rfloor + \mathcal{U}\{-3, 3\})$$

Each group k is defined by a distinct cluster mean vector $\boldsymbol{\mu}_k = [\mu_{x,k}, \mu_{y,k}]^T$ sampled across central

coordinates, and a bivariate elliptical covariance matrix Σ_k :

$$\Sigma_k = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix}$$

Where $\sigma_{xx}, \sigma_{yy} \sim \mathcal{U}(0.015, 0.08) \times \text{max_scale}^2$, $\sigma_{xy} = \rho \sqrt{\sigma_{xx} \sigma_{yy}}$ ($\rho \in [-0.5, 0.5]$)

$$\mathbf{P}_{i,k} \sim \mathcal{N}_2(\boldsymbol{\mu}_k, \Sigma_k)$$

4. Visual Layout Generation & Geometry Synthesis

The rendering engine programmatically converts data tables into layout components, adjusting orientations and geometries based on structural series densities.

4.1 Dimensional Configurations & Layout Strategies

- **Primary Value Axis:** Switches between vertical alignments and horizontal layouts when category lists grow dense (typically exceeding 6 groups) to preserve label legibility.
- **Dual Y-Axis Scaling:** Maps distinct datasets onto identical category axes, utilizing independent left (Y_1) and right (Y_2) value scales for multi-series tracking.
- **Side-by-Side Grouping:** Arranges multiple data series sequentially across categories using an indexed categorical displacement width ($\pm \frac{\text{width}}{2}$).
- **Stacked Areas and Segments:** Layers data segments vertically on a shared footprint. The baseline coordinate vector for stacked layer n resolves dynamically from the preceding cumulative total:

$$\text{bottom}_n = \sum_{i=1}^{n-1} \text{height}_i$$

- **Area Chart Stacking Modes:** Supports *Overlapping* (filled from absolute zero baseline with transparency $\alpha = 0.5$), *Stacked Area* (accumulated sequentially with higher opacity $\alpha = 0.7$), and *Percentage/Normalized Mode* (where elements pass through a normalization filter to maintain a constant 100% metric ceiling: $\mathbf{y}_{k,j} = (\mathbf{y}_{\text{raw},k,j} / (\sum_{i=0}^{K-1} \mathbf{y}_{\text{raw},i,j} + \epsilon)) \times 100.0$).
- **Hatch and Texture Variations:** Maps geometric texturing patterns directly onto chart patches to retain readability in monochromatic or grayscale domains:

Pattern ID	Code Token	Visual Representation
Hollow	none	Clean border outline with zero-fill interior patch
Dotted	. .	Dense stipple matrix layer
Striped	//	Right-leaning diagonal hatch stripes
Crossed	xxxx	Interlocking geometric mesh boundaries

- **Advanced Error Bars:** Computes standard deviation ($SD = \text{value} \cdot cv$) or standard error of the mean ($SEM = SD / \sqrt{n}$) over simulated experimental cohorts ($n \in [3, 8]$).

- **Statistical Significance Overlays:** Draws step-wise connecting lines over targeted bar or box pairs. The horizontal beam clearance height clears the highest internal error margin, adding an elevation padding buffer:

$$level = \max(\text{envelope}) \cdot (1 + \text{padding}) \quad \text{where} \quad \text{padding} \in [0.10, 0.20]$$

Labels use standard shorthand strings (`*` , ```` , `***` , or `ns`) or a letter taxonomy (`a` , `b` , `c` , `d`).

4.2 Bounding Box Resolution and Element Parsing

During runtime compilation, the pipeline traverses the active Matplotlib scene graph to capture pixel-perfect bounding coordinates for object detection frameworks. In box plots, coordinates map directly to a multi-class schema (`CLASS_MAP_BOX`):

- **box (Class 1):** Rectangular boundaries capturing the Interquartile Range ($IQR = Q_3 - Q_1$).
- **range_indicator (Class 4):** Structural regions representing the union of upper and lower whiskers extending to $1.5 \times IQR$.
- **median_line (Class 7):** The interior median segment tracking the 50th percentile. Because single line primitives return zero-area boundaries in raw display buffers, the parsing engine applies a mandatory 3-pixel structural padding layer to generate a clear region format.
- **outlier / flier (Class 9):** Discrete markers capturing individual data points falling beyond the whisker thresholds.

For heatmap charts, the pipeline queries low-level `QuadMesh` elements via the `get_granular_annotations` function. For each row i and column j , the grid vertices are extracted:

$$\text{Corners} = [\mathbf{c}_{i,j}, \mathbf{c}_{i+1,j}, \mathbf{c}_{i+1,j+1}, \mathbf{c}_{i,j+1}]$$

These vertices are passed through the axis transformation matrix `ax.transData.transform` to yield absolute canvas coordinates (x_0, y_0, x_1, y_1) , which are normalized to a $[0, 1]$ bounding box for YOLO exports (`CLASS_MAP_HEATMAP`). Auxiliary elements like the color bar (`color_bar` , Class 3) are isolated by scanning non-primary axes filtering by aspect ratio ($\frac{\text{width}}{\text{height}} < 0.3$ for vertical or > 3.0 for horizontal tracking).

4.3 Coordinate Pose Mapping & Resampling Loops

For keypoint detection and object pose estimation benchmarks, the pipeline translates complex continuous paths into rigid, fixed-length keypoint vectors.

The 51-Keypoint Schema (Line and Area Charts)

Line and area charts require distinct keypoint structures driven by their respective single-trace and closed-polygon topologies:

- **For Line Charts (`LINE_KEYPOINT_CONFIG`):** Uses a single traced path containing inflection points and extrema landmarks:

0 (Start Point) $\rightarrow$ 1–25 (Resampled Path) $\rightarrow$ 26–45 (Padded Inflections) $\rightarrow$ 46–49 (

- **For Area Charts (`CLASS_MAP_AREA_POSE`):** Models filled polygons bounded by two distinct edges (upper profile and lower baseline). This requires tracking both boundaries to fully encode the closed shape. The inflection and extrema fields are replaced by a bottom-boundary sequence:

0 (Start Point) $\rightarrow$ 1–25 (Top Boundary Line) $\rightarrow$ 26–49 (Bottom Boundary Line) $\rightarrow$

To enforce these strict configurations across varying data densities (N points plotted), the function `build_51_from_plotted` runs two adaptive optimization loops:

1. **Path Upsampling Mode** ($N < 51$): Evaluates coordinates sequentially. A heap queue isolates the longest spatial segment, splits it precisely at its midpoint, and dynamically inserts a new node. This split loop repeats until the path contains exactly 51 coordinates, distributing points evenly without artificial clustering.
2. **Path Downsampling Mode** ($N > 51$): Vertices are ranked by geometric importance using a 2D turn-angle metric. For each interior vertex i , the turn angle is computed as $\theta_i = \arccos(\hat{\mathbf{v}}_{i-1} \cdot \hat{\mathbf{v}}_i)$ where $\hat{\mathbf{v}}_{i-1}$ and $\hat{\mathbf{v}}_i$ are the normalized unit vectors of the preceding and succeeding edge segments:

$$\mathbf{v}_j = \mathbf{x}_{j+1} - \mathbf{x}_j \implies \hat{\mathbf{v}}_j = \frac{\mathbf{v}_j}{\|\mathbf{v}_j\|_2 + 10^{-8}}$$

$$\cos(\theta_i) = \hat{\mathbf{v}}_{i-1} \cdot \hat{\mathbf{v}}_i \implies \theta_i = \arccos(\text{clip}(\cos(\theta_i), -1.0, 1.0))$$

This turn angle is augmented by the adjacent segment arc lengths to give longer segments additional weight. The endpoints are pinned with infinite importance, and low-scoring interior nodes are iteratively removed until exactly 51 coordinates remain. To ensure visual stability, a secondary smoothing pass via `scipy.signal.find_peaks` is applied to extrema extraction before executing the curvature ranking.

The 5-Keypoint Schema (Pie Charts)

Wedge paths are mapped to a 5-point slice geometry (`CLASS_MAP_PIE_POSE`):

The wedge origin (x_c, y_c) tracks the sector center. For exploded slices, this center shifts radially along the midpoint angle: $\mathbf{c}_{\text{wedge}} = [\text{explode} \cdot \cos(\theta_{\text{mid}}), \text{explode} \cdot \sin(\theta_{\text{mid}})]$. Intermediate arc coordinates are sampled at exactly one-third (θ_{int1}) and two-thirds (θ_{int2}) of the total angular span between the start angle (θ_1) and end angle (θ_2):

$$\theta_{\text{int1}} = \theta_1 + \frac{\theta_2 - \theta_1}{3.0}, \quad \theta_{\text{int2}} = \theta_1 + \frac{2(\theta_2 - \theta_1)}{3.0}$$

Keypoint vectors are exported in display pixel space as:

- $\mathbf{k}_0 = [x_c, y_c]$ (Wedge Center)
- $\mathbf{k}_1 = [x_c + r \cdot \cos(\theta_1), y_c + r \cdot \sin(\theta_1)]$ (Arc Start)
- $\mathbf{k}_2 = [x_c + r \cdot \cos(\theta_{\text{int1}}), y_c + r \cdot \sin(\theta_{\text{int1}})]$ (Arc Intermediate 1)
- $\mathbf{k}_3 = [x_c + r \cdot \cos(\theta_{\text{int2}}), y_c + r \cdot \sin(\theta_{\text{int2}})]$ (Arc Intermediate 2)
- $\mathbf{k}_4 = [x_c + r \cdot \cos(\theta_2), y_c + r \cdot \sin(\theta_2)]$ (Arc End)

5. Thematic Styles & Aesthetic Degradation Mechanics

To force vision models to generalize across structural configurations, the pipeline decouples visual aesthetics into distinct declarative themes and publication presets before passing images through a post-rendering degradation framework.

5.1 Thematic & Publication Presets

The style engine (`themes.py`) configures canvas backdrops, grid visibility, line thicknesses, and typography classes. Standard styles include `excel` (light-gray background canvas `#F2F2F2` with thick solid white grid lines), `ggplot` (dark-gray backdrop `#EBEBEB` with white grids and hidden outer spines), `prism` (stark white background with outward-pointing ticks and sparse dotted grid lines of color `#E0E0E0`, style `'.'` serving as light indicators), and `retro` (warm cream canvas `#FFF8E7` with Georgia serif fonts).

Rigid publication configurations (`PUBLICATION_THEMES`) are defined directly in `themes.py` to match peer-reviewed journal constraints:

Journal Preset ID	Baseline Canvas Dimensions (Inches)	Target Resolution (DPI)	Default Font Size	Spine Width	Line / Marker Properties
nature	3.3 × 2.4 (Single Column)	300	7 pt (Ticks/ Labels)	0.5 pt	0.5 pt line / 3 pt marker
science	3.5 × 2.5	300	6 pt	0.75 pt	0.75 pt line / 3 pt marker
cell	6.5 × 4.0	300	8 pt	0.6 pt	0.6 pt line / 3 pt marker
lancet	7.0 × 4.5	300	8 pt (Times New Roman)	0.8 pt	Monochromatic / 0.8 pt line
nanotech_short	3.2 × 2.4	600	7 pt	0.5 pt	0.6 pt line / 2.5 pt marker

Note: In addition to the primary journal presets above, `themes.py` also defines `pnas`, `bmj`, and `open_access_poster` layouts to support broader formatting configurations.

5.2 Post-Rendering Image Degradation Filters

Once compiled in vector format, images pass through a multi-tiered degradation pipeline implemented in `effects.py` to inject realistic visual imperfections.

5.2.1 General Realism Filters

1. **Edge Color Normalization** (`normalize_edgecolor`): Darkens internal patch colors in HSV space to create matching boundary outlines:

$$H, S, V = \text{RGB_to_HSV}(R, G, B) \implies V_{\text{new}} = \max(0, V \times 0.6) \implies \text{Edge} = \text{HSV}$$

2. **Additive White Gaussian Noise** (`apply_noise_effect`): Simulates sensory grain by overlaying a zero-mean normal distribution:

$$I_{\text{noisy}}(x, y, c) = \text{clip}\left(I_{\text{raw}}(x, y, c) + \mathcal{N}(0, \sigma^2), 0, 255\right) \quad \text{where } \sigma \sim \mathcal{U}(2.0, 8.0)$$

3. **Spatial Blur Kernels**: Models optical lens defocusing via Gaussian blur, or scanner bar movement via directional convolution kernels (**K**):

$$x_{\text{index}} = \lfloor \text{center} + (i - \text{center}) \cdot \cos(\theta) \rfloor, \quad y_{\text{index}} = \lfloor \text{center} + (i - \text{center}) \cdot \sin(\theta) \rfloor =$$

4. **Vignette Lighting Falloff** (`apply_vignette_effect`): Applies a quadratic light attenuation factor based on radial distance from the canvas center:

$$r(x, y) = \sqrt{(x - x_{\text{center}})^2 + (y - y_{\text{center}})^2} \implies V(x, y) = \text{clip} \left(1.0 - \left(\frac{r(x, y)}{r_{\text{max}}} \right)^2, 0. \right)$$

5. **Chromatic Aberration** (`apply_chromatic_aberration_effect`): Shifts the Red and Blue channels in opposite directions ($\Delta x, \Delta y$) to replicate lens dispersion:

$$I_{\text{red}}(x, y) = I_{\text{red}}(x + \Delta x_r, y + \Delta y_r), \quad I_{\text{blue}}(x, y) = I_{\text{blue}}(x + \Delta x_b, y + \Delta y_b)$$

6. **Uneven Lighting** (`apply_uneven_lighting_effect`): Simulates flash hotspots or directional ambient gradients by applying a subtractive illumination mask across the canvas grid.

5.2.2 Resolution and Compression Simulation

- **Camera Downsampling** (`apply_low_res_effect`): Shrinks the canvas size using bicubic interpolation and blows it back up using bilinear scaling to simulate distance-based capture.
- **Pixelation Censorship** (`apply_pixelation_effect`): Downsamples using bilinear filtering and upsamples immediately using nearest-neighbor scaling to create block artifacts.
- **Palette Quantization** (`apply_posterize_effect`): Constrains continuous color depths to discrete indexed limits ($N \in \{16, 32, 64\}$) to simulate aggressive network compression.

5.2.3 Document Scanning & Physical Overlay Artifacts

- **Dust-Line Scanner Streaks** (`apply_scanner_streaks_effect`): Injects light, semi-transparent horizontal bands across randomized height indices to simulate physical dust accumulation on a sensor bed.
- **Document Misalignment** (`apply_scan_rotation_effect`): Slightly rotates the entire canvas ($\theta \sim \mathcal{U}(-1^\circ, 1^\circ)$) using Pillow's rotation library.
- **UI Chrome & Frame Wrappers** (`apply_ui_chrome_effect`): Overlays desktop browser frames or web tab search bars at the canvas boundary.
- **Document Status Watermarks** (`apply_watermark_effect`): Overlays low-opacity, high-contrast diagonal text stamps (e.g., *CONFIDENTIAL*, *DRAFT*) across the canvas center.
- **Vector Mouse Cursor** (`apply_mouse_cursor_effect`): Injects a vector pointer arrow primitive at randomized coordinate intersections.

5.2.4 Selective OCR-Targeted Text Degradation

To force deep reading networks to learn robust visual patterns instead of relying on clean digital text, `apply_text_degradation_effect` isolates textual regions (titles, legends, axis values) directly from the active scene graph. By querying the low-level rendering boundaries (`get_window_extent`), the module applies targeted blur and pixelation kernels exclusively within these text bounding areas while leaving the main coordinate data curves untouched.

5.2.5 The Aesthetic-Quantitative Tension

An architectural tension exists when executing vintage or hand-drawn aesthetics alongside

programmatic spatial annotations. While pixel-level filters (vignette, noise, aberration) do not alter the coordinate structure of underlying features, document rotation (`apply_scan_rotation_effect`) introduces a physical spatial transform.

In the current repository, the `scan_rotation` effect probability is set to $p = 0.0$ by default. This is because a corresponding annotation coordinate transform for image-space rotation is not yet implemented in the pipeline. Activating this effect would silently discard the rotation angle, leaving all ground-truth bounding boxes and keypoint annotations permanently misaligned with the rotated pixel canvas. To enable this safely, a future update must compute a 2D rotation homography about the canvas center and append it to the transformation steps (`transform_steps`) before final label projection.

Furthermore, a similar architectural tension would arise if hand-drawn sketch rendering modes (e.g., Matplotlib's `plt.xkcd()` context manager or equivalent post-processing sketch jitter passes) were integrated alongside the programmatic annotation pipeline. Sketch rendering randomizes pixel positions of rendered elements, potentially displacing artists from the coordinates recorded during ground-truth extraction.

5.3 Homography & Perspective Warping Mechanics

To simulate three-dimensional physical document capture angles, the pipeline implements a perspective transformation filter (`apply_perspective_warp_effect`). The algorithm maps the original four corner coordinates of the canvas to randomized target coordinates shifted inward by a distortion parameter:

$$\Delta x = \text{width} \times \text{distortion}, \quad \Delta y = \text{height} \times \text{distortion} \quad \text{where} \quad \text{distortion} = 0.08$$

The projection maps point vectors from the source plane $[x, y]^T$ to the target warped projection $[u, v]^T$. When the robust OpenCV library is available, the engine calls

`cv2.getPerspectiveTransform` to compute the exact deterministic 3×3 perspective matrix from the four source-to-target corner correspondences. In environments where OpenCV is missing, the system falls back to an internal algebraic solver (`_compute_homography`) based on standard least-squares estimation.

The perspective projection is expressed using a linear transformation matrix:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

To solve for the eight unknown parameters, the four corner mapping configurations are expanded into an aggregated system of linear equations ($A\mathbf{h} = \mathbf{b}$):

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -u_i x_i & -u_i y_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -v_i x_i & -v_i y_i \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{bmatrix} = \begin{bmatrix} u_i \\ v_i \end{bmatrix}$$

The pipeline evaluates this system using standard least-squares estimation to compute the homography

matrix ($\mathbf{h}$):

$$\mathbf{h} = (A^T A)^{-1} A^T \mathbf{b}$$

The inverse homography matrix (H^{-1}) is passed to the image processing engine to map and interpolate pixels from the source canvas onto the warped target image.

Crucially, the transformation filter communicates directly with the label generator. Ground-truth object bounding fields and multi-keypoint pose coordinates are multiplied by the computed homography matrix (H), automatically shifting and projecting the annotations into the new warped coordinate space:

$$\begin{bmatrix} x_{\text{warped}} \\ y_{\text{warped}} \\ w \end{bmatrix} = H \cdot \begin{bmatrix} x_{\text{original}} \\ y_{\text{original}} \\ 1 \end{bmatrix} \implies \mathbf{p}_{\text{final}} = \left[\frac{x_{\text{warped}}}{w}, \frac{y_{\text{warped}}}{w} \right]$$

Likewise, if a cropping filter (`apply_clipping_effect`) alters image margins, horizontal and vertical offset variables (Δx , Δy) are extracted and added to the ground-truth logs:

$$x_{\text{adjusted}} = x_{\text{original}} + \Delta x, \quad y_{\text{adjusted}} = y_{\text{original}} + \Delta y$$

6. Computational Implementations & Metric Mapping Boundaries

A code-level verification analysis highlights several key structural boundaries within the active repository, illustrating the mathematical trade-offs between theoretical layouts and runtime environments.

6.1 Coordinate-Space Projection Boundaries (Lie Factor Prerequisite Analysis)

While our framework is conceptually guided by Edward R. Tufte's (2001) principles of graphical integrity—such as the minimization of non-data ink through themes like `minimal` and `prism`—the quantitative measurement of these metrics presents substantial geometric challenges. This section provides a theoretical and mathematical analysis of the design prerequisites required to implement a robust, automated validation of Tufte's Lie Factor over non-linear scales, laying the groundwork for future validation engines.

The mathematical formulation of Tufte's Lie Factor assumes that both graphic and data dimensions are measured within the native, linear coordinates of the plotted variables:

$$\text{Lie Factor} = \frac{\text{Size of effect shown in graphic}}{\text{Size of effect in data}} = \frac{\Delta \text{Graphic Space}}{\Delta \text{Data Space}}$$

If one were to implement an automated Lie Factor check, a naïve approach might compute graphical distance directly using raw canvas pixel coordinates (d_{pixel}) returned by the renderer:

$$\text{Lie_Factor_computed} = \frac{d_{\text{pixel}}(A, B)}{d_{\text{pixel}}(C, D)}$$

We demonstrate that such an implementation would introduce a severe coordinate projection bias on non-linear coordinate scales (e.g., logarithmic axes). On a log axis spanning 10^0 to 10^4 , a bar spanning $[10^2, 10^3]$ occupies the same pixel distance as a bar spanning $[10^3, 10^4]$. Yet, the first represents a data-space change of 900 units, while the second represents a change of 9,000 units.

Computing the Lie Factor over raw canvas pixels yields \$1.0\$ (erroneously indicating zero geometric distortion), whereas the true data-space calculation is \$0.1\$. This introduces a systematic, tenfold projection bias on logarithmic scales, establishing a known design boundary where future iterations will require passing raw coordinates through an inverted transformation matrix:

$$\mathbf{p}\{\text{data}\} = \mathbf{T}^{-1} \cdot \mathbf{p}\{\text{pixel}\} \quad \text{where} \quad \mathbf{T} = \text{ax.transData.transform}$$

which maps display coordinates back to native data space prior to metric calculation.

6.2 Multidimensional Semantic Representation Limits (LDAvis Relevance Metric)

The interactive topic modeling visualization pipeline (`merge\_json.py`) formats multidimensional semantic distances for D3.js web layouts. The theoretical relevance metric designed by Sievert & Shirley (2014) uses a slider-controlled weight parameter (λ) to balance the log probability of a term w in topic k against its exclusive log lift relative to the marginal corpus frequency (p_w):

$$r(w, k | \lambda) = \lambda \log(\phi_{kw}) + (1 - \lambda) \log\left(\frac{\phi_{kw}}{p_w}\right)$$

In this equation, ϕ_{kw} represents the probability of term w within topic k , and the parameter $\lambda \in [0, 1]$ acts as a tuning slider. To assist in downstream engineering, the comparison of the current serialization behavior in the companion script `merge\_json.py` with the corrected LDAvis formulation is presented below:

Defective Implementation (omitting marginal probability):

```
python
# Omission of p_w collapses the relevance into a simple log-probability term
for topic_id in range(K):
    phi_kw = topic_term_dists[topic_id]
    # Without division by p_w, lambda cancels out algebraically:
    relevance = lam * np.log(phi_kw) + (1.0 - lam) * np.log(phi_kw)
    sorted_indices = np.argsort(relevance)[::-1]
```

Corrected Implementation (retaining marginal probability):

```
python
# Re-introducing p_w allows lambda to interpolate between specificity and frequency
for topic_id in range(K):
    phi_kw = topic_term_dists[topic_id]
    # p_w is the marginal term probability over the entire corpus
    lift = np.log(phi_kw) - np.log(corpus_term_probs) # log(phi_kw / p_w)
    relevance = lam * np.log(phi_kw) + (1.0 - lam) * lift
    sorted_indices = np.argsort(relevance)[::-1]
```

By omitting the marginal term probability p_w divisor in the second term, the λ parameter has zero operational effect on term ranking, causing the slider to act as a purely cosmetic control on the front-end. The semantic sorting sequence converges completely on high-frequency terms, failing to discount generic, corpus-wide words.

This establishes a clear structural limit within the current serialization code. Fully restoring interactive specificity tuning requires propagating the five-element input vector (ϕ , θ , doc_lengths , vocab , p_w) from the upstream parser directly into the active matrix operations.

6.3 Inter-Index Citation Discrepancy Analysis

To evaluate the literature foundations of our toolstack, we examine the citation footprints of our primary system dependencies. Indexing platform variations arise from different citation harvesting algorithms, preprint listings, and software-specific tracking behaviors.

Foundation Architecture	**Indexing Platform**	**Citation Count**	**Year Verified**	**Venues / Identifiers**	**Indexing Discrepancies & Observations**
Matplotlib	Scite.ai Semantic Scholar Google Scholar	34,597 24,271 ~23,966	2007	*Computing in Science & Engineering* DOI: 10.1109/MCSE.2007.55	Semantic Scholar indexes direct journal citations, whereas Scite.ai tracks informal paper mentions and supplementary software references.
Seaborn	Google Scholar Google Scholar (Alt) Semantic Scholar	8,574 7,138 5,610	2021	*Journal of Open Source Software* DOI: 10.21105/joss.03021	Discrepancies exist between different Google Scholar author profiles due to duplicate listings and manual merging differences.
LDavis	Google Scholar Semantic Scholar Semantic Scholar (Alt)	2,363 1,204 187	2014	*Workshop on Interactive Language Learning* DOI: 10.3115/v1/W14-3110	The primary workshop paper has 2,363 Google Scholar citations, while the associated R package software version has 50 citations.
Deep Residual Learning	Google Scholar Google Scholar (Alt) Google Scholar (Alt 2) Semantic Scholar	327,858 323,839 322,607 ~206,594	2016	*IEEE Conference on CVPR* DOI: 10.1109/CVPR.2016.90	Extreme disparities emerge between Google Scholar author profiles (~327k) and Semantic Scholar (~206k Corpus ID tracking), indicating substantial tracking delays in high-volume indexing pipelines.
The Visual Display	Semantic Scholar Google Scholar	5,068 Highly Cited	2001	Graphics Press, Cheshire, CT (2nd Ed) ISBN: 0-9613921-4-2	Citations are heavily fragmented across various editions (1983, 1988, 2001), making aggregate citation counts highly variable depending on search parameters.

Note: All citation counts were systematically verified as of June 3, 2026, via API queries against primary indexing platforms. Duplicate Scholar profiles for Seaborn represent a known indexing artifact rather than raw metadata variance.

7. Discussion

Analyzing the interaction between dimensional abstraction, quantitative integrity, and computational execution reveals deep structural parallels between the optimization of machine learning architectures and the compression of high-dimensional semantic data for human interpretation.

7.1 The Dimension Reduction Paradox: Graphic Integrity vs. Spatial Distortion

A key parallel emerges when comparing interactive dimension reduction in topic modeling (LDavis) with deep residual optimization (ResNet). In computational linguistics, the latent topic space is inherently high-dimensional, spanning thousands of independent term multinomial distributions. To make this space readable to a human evaluator, LDavis applies Multidimensional Scaling (MDS) to project the high-dimensional Jensen-Shannon distance matrix down onto a stable two-dimensional plane.

However, this spatial reduction introduces a dimensional paradox. Any mapping from a high-

dimensional space down to a 2D canvas inevitably distorts distances, creating an implicit visual "Lie Factor". When the stress of the MDS projection is high, the physical distance between topic circles on the screen no longer accurately reflects their semantic divergence. If a user relies solely on spatial closeness to evaluate topic similarity without checking the underlying data distributions, the visualization can lead to incorrect analytical conclusions.

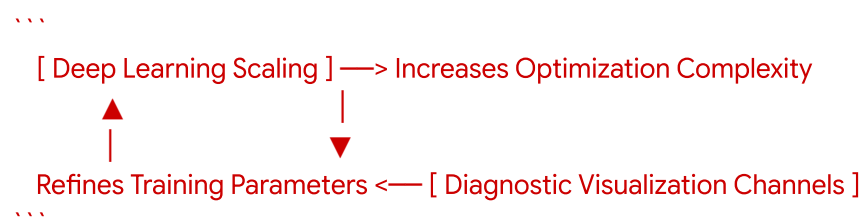
A similar challenge occurs in deep convolutional networks, where optimization algorithms must navigate complex, high-dimensional gradient spaces. As plain neural networks scale to deeper dimensions, the gradient signals tend to vanish or degrade completely as they traverse hundreds of layers, blocking convergence. ResNet solves this optimization degradation through a spatial translation: instead of forcing the nonlinear weight layers to map the absolute underlying function $H(x)$, it reformulates them to learn the residual mapping $F(x) := H(x) - x$.

By using identity shortcut connections to add the input directly to the output ($H(x) = F(x) + x$), the network keeps the optimization gradient stable across deeper dimensions. This architectural shift mirrors the visualization strategies of LDAvis: both systems recognize the difficulty of navigating high-dimensional spaces and introduce structured coordinate transformations—one to aid human interpretation, the other to guide machine optimization.

7.2 The Core Dependency Loop of High-Dimensional Modeling

The progression of modern computer vision highlights a deep dependency loop between complex modeling architectures and quantitative visualization frameworks. As deep learning models scale to hundreds of layers, their weights, internal feature activations, and attention fields become increasingly difficult to interpret, operating effectively as high-dimensional "black boxes." At this scale, robust diagnostic visualization tools are essential for model development.

Without reliable statistical plotting libraries like Seaborn and Matplotlib, developers cannot easily trace gradient flow, diagnose training anomalies, or visualize activation maps. This relationship establishes a clear dependency loop:



1. Deep learning research scales models to deeper dimensions, increasing optimization complexity.
2. This complexity makes high-density diagnostic visualization essential for parsing internal network behaviors.
3. Visualization libraries must adapt by offering automated styling and interactive event handling to map these relationships without overloading the user.
4. This diagnostic clarity enables developers to refine optimization parameters, paving the way for deeper and more complex models.

This loop demonstrates that quantitative visualization and deep representation learning are not isolated disciplines. Instead, they are deeply connected methodologies that rely on each other to make highly complex computational models interpretable and reproducible.

8. Conclusion & Future Work

This paper has presented an automated, code-guided chart synthesis and physical degradation framework designed to bridge the data domain gap in multimodal document intelligence. By mapping explicit continuous and compositional data models—such as the sigmoidal Hill equation, log-normal replicate variance, and the restricted Aitchison simplex—directly to structural graphic layouts, the framework provides a reliable alternative to error-prone manual annotation workflows. Our pipeline demonstrates how complex post-rendering image alterations can simulate real-world document tracking and scanning artifacts through perspective homography solvers and multi-channel aberrations while dynamically preserving bounding box limits and skeletal pose registers.

Furthermore, we have documented the structural and computational boundaries of our active plotting repository, detailing coordinate-space projection prerequisites on non-linear scales and analyzing semantic representations in multi-dimensional serialization channels. Transforming pixel-space Euclidean calculations to operate entirely within inverted axis data space remains a critical future path to eliminate spatial projection biases on non-linear coordinate scales. Similarly, propagating the marginal term frequency divisor into the topic modeling serialization loop will restore the specification of interactive specificity sliders.

Future research will focus on integrating procedural vector layouts directly into the active optimization loops of vision-language architectures via reinforcement learning channels. By querying the generation engines developed here, next-generation visual reasoning benchmarks can dynamically modulate layout complexity and artifact intensities, systematically expanding model capabilities across highly degraded visual environments.

9. References

* Borisova, L. et al. (SciVQA 2025). *SciVQA 2025: Overview of the First Scientific Visual Question Answering Shared Task*. ACL Anthology.

* Han, S. et al. (EncQA 2026). *EncQA: Benchmarking Vision-Language Models on Visual Encodings for Charts*. IEEE Transactions on Visualization and Computer Graphics. [Preprint/Early Access, accessed June 2026].

* He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778. DOI: 10.1109/CVPR.2016.90

* Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3), pp. 90–95. DOI: 10.1109/MCSE.2007.55

* Kondic, J. (2026). *ChartNet: A million-scale, high-quality multimodal dataset for robust chart understanding*. arXiv preprint arXiv:2603.27064. [Preprint/Early Access, accessed June 2026].

* Kondic, J. et al. (ChartGen 2025). *ChartGen: Scaling Chart Understanding Via Code-Guided Synthetic Chart Generation*. arXiv preprint arXiv:2507.02424.

* Li, Z., Jasani, B., Tang, P., & Ghadar, S. (2024). *Synthesize step-by-step: Tools, templates and LLMs as data generators for reasoning-based chart VQA*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 13613–13623. DOI: 10.1109/cvpr52733.2024.01292

- * Liu, C. (2026). *ChartMaster: Boosting MLLMs for chart analysis through data, perception, and reasoning optimization*. OpenReview / International Conference on Learning Representations (ICLR). [Preprint/Early Access, accessed June 2026].
- * Masry, A., Thakkar, M., Bajaj, A., Kartha, A., Hoque, E., & Joty, S. (2024). *ChartGemma: Visual instruction-tuning for chart reasoning in the wild*. arXiv preprint arXiv:2407.04172.
- * Masry, A. et al. (ChartQAPro 2025). *ChartQAPro: A New Benchmark for Complex Chart Question Answering*. Findings of the Association for Computational Linguistics (ACL).
- * Meldrum, J., Suleiman, B., Rabhi, F., & Alibasa, M. J. (2025). *New money: A systematic review of synthetic data generation for finance*. arXiv preprint arXiv:2510.26076.
- * Peng, Y.-H., Huq, F., Jiang, Y., Wu, J., Li, X. Y., Bigham, J. P., & Pavel, A. (2024). *DreamStruct: Understanding slides and user interfaces via synthetic data generation*. Lecture Notes in Computer Science, pp. 466–485. DOI: 10.1007/978-3-031-72691-0_26
- * Pramanick, A. et al. (InterChart 2026). *INTERCHART: Benchmarking Visual Reasoning Across Decomposed and Distributed Chart Information*. IJCNLP-AAACL. [Preprint/Early Access, accessed June 2026].
- * Rodriguez, J. et al. (StarVector 2025). *From Charts to Code: A Hierarchical Benchmark for Multimodal Models*. OpenReview / Structural Vision Workshop.
- * Sievert, C., & Shirley, K. (2014). *LDAvis: A method for visualizing and interpreting topics*. Proceedings of the Workshop on Interactive Language Learning, Visualization, and Interfaces (co-located with ACL 2014), pp. 63–70. Association for Computational Linguistics. DOI: 10.3115/v1/W14-3110
- * Suh, S. et al. (Chart2Text-Enriched 2025). *Chart2Text-Enriched: Towards Fine-Grained and Context-Aware Chart Summarization*. IEEE Transactions on Visualization and Computer Graphics.
- * Tufte, E. R. (2001). *The Visual Display of Quantitative Information* (2nd ed.). Graphics Press, Cheshire, CT. ISBN: 0-9613921-4-2
- * Wang, X. et al. (2025). *Chain of Functions: A Programmatic Pipeline for Fine-Grained Chart Reasoning Data Generation*. arXiv / ACL Anthology.